

Business Signal

Ask a business question. Find the signal.

An end-to-end AI engineering project that answers natural-language business questions with structured evidence, SQL intelligence, retrieval-augmented generation and deterministic risk analysis.

FastAPI

PostgreSQL

pgvector

Next.js

Docker

Gemini AI

RAG

Sentence-Transformers

20 + Pages

Documentation

8 + Modules

Backend

21 / 21

Tests Passing

4 / 4

Evaluations

Ranjith S

AI Engineering · Full-Stack Development · GenAI Applications

github.com/ranjiths112007/Business-Signal-enterprise-ai

September 2026

CONTENTS

Table of Contents

01	Executive Summary	3
02	Project Overview	4
03	Architecture Deep Dive	5
04	Technology Stack	7
05	Backend Engineering	9
06	AI & Intelligence Layer	12
07	Frontend Application	15
08	Database Design	16
09	API Reference	17
10	Security Engineering	18
11	Testing & Evaluation	19
12	Data Ingestion Pipeline	20
13	Deployment Architecture	21
14	Performance & Scalability	22
15	Future Roadmap	23
16	Conclusion	24

Executive Summary

Business Signal is an end-to-end AI engineering project I built to demonstrate how structured business data, a natural-language SQL agent, retrieval-augmented generation (RAG), and deterministic risk analysis can be combined into a single coherent intelligence pipeline — one that always shows its working and never makes up facts.

The project answers real business questions — *Which customers are at risk? What is our total revenue? Which industry drives the most growth?* — by routing each question to the right source: a SQL query, a vector similarity search, or a deterministic rule engine. Every answer is returned alongside the exact evidence that produced it.

This document covers the full technical design, every module, the AI engineering decisions, test coverage, security model, and deployment approach. It is written as a technical showcase of what I built, how I built it, and why each decision was made.

1,310 Customers in demo DB	■ 1.6M+ Demo revenue tracked	21 / 21 Unit tests passing	4 / 4 Eval tests passing
--	--	--	--

What makes this an AI Engineering project — not just a CRUD app:

- **Grounded answers:** every response cites structured evidence — no hallucinations, no invented numbers.
- **Multi-source routing:** questions are classified then sent to SQL, retrieval, or rules based on intent.
- **No-LLM fallback:** the system answers 16+ question categories deterministically without any API key.
- **RAG pipeline:** PDF documents are chunked, embedded with Sentence-Transformers, and stored in pgvector.
- **SQL agent:** natural-language questions are converted to safe, read-only PostgreSQL SELECT queries.
- **Prompt-injection guard:** every user question is sanitized before reaching the LLM.
- **Risk analysis engine:** each customer gets a scored risk level driven by revenue trend and ticket data.
- **Smart CSV mapper:** uploads fuzzy-match 40+ common column name variants — no manual mapping required.

Project Overview

Background & Motivation

I built Business Signal to answer a practical question: *what does a well-engineered AI system that answers structured business questions actually look like under the hood?* Not a chatbot that confidently makes things up, but a system with a deliberate pipeline — classify, retrieve, reason, cite. The goal was to build something small enough to fully understand end-to-end while demonstrating real AI engineering depth.

Core Problem Statement

Business analysts spend hours manually querying databases and spreadsheets. Generic chatbots hallucinate numbers. Business Signal asks: can we build a system that **answers naturally, reasons correctly, and always shows its evidence?**

The Answer Pipeline

01	Question In	User asks in plain English through the web UI
02	Intent Classify	System determines: business data, document, or general query
03	Source Selection	Routes to SQL agent, vector search, or rule engine
04	Evidence Build	Gathers structured facts: revenue, risk scores, ticket counts
05	LLM Synthesis	Gemini drafts a grounded answer from the evidence context
06	Fallback Mode	Without an API key: deterministic rules answer 16+ categories
07	Answer + Trace	Response returned with full evidence trace attached

Demo Dataset

The project ships with a deterministic seeded dataset designed to produce interesting and realistic signal from the analysis pipeline:

Entity	Count	Key Detail
Customers	1,310	Across 14 industries including SaaS, Fintech, Healthcare, Energy

Sales records	10,000+	Jan–Aug 2026, realistic seasonal patterns
Support tickets	14 open	2 high-priority, 12 medium — deliberately skewed for demo
At-risk accounts	2 HIGH	Apex Logistics (–48.6% revenue), Pulse Media (–34.1%)
Industries	14	Fintech, Energy, Logistics, Manufacturing, Retail, SaaS and more

Architecture Deep Dive

Business Signal follows a layered architecture where each layer has a single, well-defined responsibility. The design is intentionally simple — no microservices, no message queues — because the goal is clarity and demonstrability, not scale.

System Layers

Presentation Layer	Next.js 14	React server/client components, CSS variables, real-time fetch
API Gateway Layer	FastAPI (Python 3.11)	Auto-documented REST API, Pydantic validation, CORS middleware
Intelligence Layer	Decision + SQL + RAG	Intent classification, SQL agent, vector retrieval, risk engine
Persistence Layer	PostgreSQL 17 + pgvector	Relational schema + vector index for document embeddings
Embedding Layer	Sentence-Transformers	all-MiniLM-L6-v2 model (384-dim) for semantic document search
LLM Layer (optional)	Google Gemini	Gemini-3.6-flash for natural language synthesis from evidence
Container Layer	Docker Compose	Reproducible dev environment: Postgres + API in containers

Request Flow Diagram (Text)

```
Browser (localhost:3050)
├─ HTTP POST /api/v1/business/ask { question: '...' }
└─
FastAPI (localhost:8000)
├─ 1. sanitize_question() ← prompt-injection guard
├─ 2. classify_question() → 'business' | 'document' | 'general'
├─
├─ [business] build_business_context()
├─   ── SELECT summary, top customers, support breakdown, industry
├─   ── customer_risk() per customer ← risk score + level
├─   ── → structured evidence dict
├─
├─ [document] search_documents()
├─   ── encode query → vector (384-dim)
├─   ── pgvector cosine search over documents table
├─   ── → top-K passages with similarity scores
├─
├─ 3. LLM_API_KEY present?
├─ YES → Gemini.generate_content(evidence + question)
├─ NO  → deterministic_fallback(evidence, question)
├─
└─ { answer, evidence, confidence } → Browser renders + scrolls to answer
```

Key Architectural Decisions

Evidence-first design

The LLM never queries the database directly. It receives a pre-built evidence dictionary from the deterministic layer. This eliminates hallucination of facts and makes every answer auditable.

Deterministic fallback

The system works fully without an LLM API key. 16+ question categories are answered correctly from structured evidence using pattern matching and data aggregation. This makes the project runnable and demonstrable even in offline/restricted environments.

Stateless API

No session state is held in the backend. Each request is fully self-contained. This simplifies scaling and makes the system trivially testable.

Docker-first development

PostgreSQL and the API run in Docker containers. The frontend runs locally via npm. This avoids environment-specific issues while keeping the dev loop fast.

Technology Stack

Every technology in Business Signal was chosen deliberately. Here I explain not just what I used, but why.

Backend & API

FastAPI 0.115+ Python async web framework	Chosen for automatic OpenAPI documentation, async support, Pydantic validation, and its native fit with modern Python type hints. FastAPI's dependency injection makes CORS, auth, and middleware trivial to configure.
Python 3.11 Runtime	3.11 gives 10–60% speed improvements over 3.9 via specialising adaptive interpreter. Used in Docker (Python 3.11-slim image) and locally on Python 3.12.
Pydantic v2 Data validation	All API request/response bodies validated via Pydantic models. Field constraints (min_length, ge) enforce correctness at the boundary.
Uvicorn ASGI server	Production-grade ASGI server with uvloop for event loop acceleration. Runs inside Docker exposing port 8000.

Database & Vector Store

PostgreSQL 17 Primary database	Battle-tested relational database for customers, sales, and support_tickets. Used with the pgvector extension to also serve as the vector store.
pgvector Vector similarity search	Official PostgreSQL extension enabling vector columns and cosine/inner-product similarity search. Stores 384-dimensional sentence embeddings for document RAG. Eliminates the need for a separate vector database like Pinecone or Weaviate.
psycopg3 Database driver	Modern async-compatible PostgreSQL driver. Binary protocol, connection pooling, and proper Decimal/date type handling. Replaced psycopg2 in this project.

AI & Machine Learning

Sentence-Transformers Embedding model	all-MiniLM-L6-v2 produces 384-dimensional semantic embeddings. Runs locally — no API call required. Used for RAG document ingestion and query encoding.
Google Gemini LLM (optional)	gemini-3.6-flash via google-genai SDK. Used when LLM_API_KEY is configured. Receives a pre-built evidence context — it synthesises, not retrieves.

pypdf PDF processing	Extracts text from uploaded PDF documents page by page. Text is chunked with 900-character windows and 120-character overlap before embedding.
--------------------------------	--

Frontend

Next.js 14 React framework	App Router with client components for interactive state. Server-side rendering for SEO. Environment variables (NEXT_PUBLIC_*) baked at build time.
Vanilla CSS Styling	CSS custom properties for the full design system. No Tailwind, no CSS-in-JS. Dark-mode neon aesthetic with glassmorphism, smooth transitions, and responsive grid layouts.
TypeScript Type safety	Full type annotations on all API response shapes, component props, and state. Catches mismatches between API contract and UI at compile time.

Infrastructure & Tooling

Docker Compose Container orchestration	Two-service compose file: postgres (pgvector image) + api (custom Python image). Health checks ensure API waits for Postgres before starting.
GitHub Actions CI/CD	Automated test pipeline runs on every push. Executes the full pytest suite and the evaluation harness against a real Postgres instance.
pytest Testing framework	21 unit tests across prompt injection, SQL security, data API, and intelligence modules. Separate evaluation harness with 4 deterministic risk assessments.

Backend Engineering

Module Structure

Module	Role	Responsibility
main.py	Application entry	FastAPI app, CORS, router registration, health setup
config.py	Configuration	Settings dataclass reading from env vars (DATABASE_URL, LLM_API_KEY, etc.)
database.py	DB connection	psycopg3 connection pool, SQLite fallback wrapper for testing
business.py	Core analytics	customer_risk() and top_customers() — pure deterministic analysis
business_api.py	REST endpoints	summary, revenue-trend, top-customers, risk, decision, sql, ask routes
decision.py	Answer engine	customer_decision(), answer(), and the _fallback() intelligence
intelligence.py	Evidence builder	classify_question(), build_business_context(), gather_evidence()
sql_agent.py	SQL generation	generate_sql() via Gemini, validate_sql() with write-op block
data_api.py	CSV ingestion	analyze, upload with fuzzy column mapping — 40+ alias variants
retrieval.py	RAG pipeline	ingest_pdf(), search_documents() using pgvector cosine similarity
prompt_guard.py	Security	sanitize_question() blocks prompt injection patterns
health.py	System health	readiness probe and metrics snapshot endpoint
metrics.py	Runtime metrics	Lightweight uptime/request counter (created this session)

Risk Scoring Algorithm

The risk engine in **business.py** is deterministic — no LLM, no probability. It calculates a 0–100 risk score for each customer using three signals:

```
# Revenue trend: compare trailing 90 days vs prior 90 days
revenue_drop = (previous - revenue) / previous * 100 if previous > 0 else 0

# Risk score formula
score = min(100, max(0,
int(revenue_drop * 1.2) # revenue decline weighted 1.2x
+ tickets * 8 # open tickets: 8 pts each
+ open_high * 12 # high-priority tickets: 12 pts each
))

# Risk level thresholds
level = 'HIGH' if score >= 60 else
'MEDIUM' if score >= 30 else 'LOW'
```

Database Connection Layer

The database module uses a context manager pattern so every connection is automatically returned to the pool. For the test suite, an in-memory SQLite wrapper implements the same interface so tests run without Docker.

```
# Production: psycopg3 with connection string from DATABASE_URL
with get_connection() as conn:
    rows = conn.execute('SELECT ...').fetchall()
    conn.commit()

# Test environment: SQLiteConnectionWrapper
# Same API, zero Docker dependency
```

API Route Design

Method	Route	Description
GET	/health	Returns service name and version
GET	/health/ready	Confirms DB connectivity
GET	/metrics	Uptime and request counters
GET	/api/v1/business/summary	Customer count, total revenue, tickets
GET	/api/v1/business/revenue-trend	Daily revenue time series
GET	/api/v1/business/top-customers	Customers ranked by revenue
POST	/api/v1/business/risk	Risk score for a specific customer_id
POST	/api/v1/business/decision/{id}	INTERVENE/MONITOR decision with reasons
POST	/api/v1/business/sql	Convert question to SQL, execute, return
POST	/api/v1/business/ask	Full intelligence pipeline answer
POST	/api/v1/data/analyze	Inspect CSV columns, suggest mapping
POST	/api/v1/data/upload	Import CSV with resolved column mapping

06 · AI & INTELLIGENCE LAYER

AI & Intelligence Layer

Question Intent Classification

Before any data is fetched, every question goes through **classify_question()** in *intelligence.py*. This lightweight function uses keyword matching to route the question to the right data source — no LLM call required.

```
def classify_question(question: str) -> str:
    q = question.lower()
    if any(x in q for x in ['policy', 'contract', 'document', 'refund', 'pdf']):
        return 'document' # → vector search over uploaded PDFs
    if any(x in q for x in ['customer', 'revenue', 'sales', 'risk', 'churn', ...]):
        return 'business' # → structured DB queries + risk engine
    return 'general' # → both sources
```

Retrieval-Augmented Generation (RAG)

The RAG pipeline in **retrieval.py** enables the system to answer questions about uploaded PDF documents. The pipeline has two phases:

Ingestion (offline)

- PDF is uploaded via the /documents endpoint
- pypdf extracts text page by page
- Text chunked: 900-char windows with 120-char overlap
- each chunk encoded → 384-dim vector via all-MiniLM-L6-v2
- Vectors stored in PostgreSQL documents table via pgvector

Retrieval (at query time)

- User question encoded to 384-dim query vector
- pgvector cosine similarity search: SELECT ... ORDER BY embedding <=> query
- Top-K passages returned (default K=5, configurable via TOP_K env var)
- Passages included in evidence context sent to LLM
- Source document and page number cited in response

Why pgvector over a dedicated vector database? At this project's scale, adding Pinecone or Weaviate would be operational overhead with no benefit. pgvector runs inside the existing PostgreSQL instance, keeps the architecture simple, and supports production workloads at millions of vectors.

Natural-Language SQL Agent

The SQL agent in **sql_agent.py** converts a natural-language question into a safe PostgreSQL SELECT query. When no LLM key is available, a keyword-based fallback generates common queries deterministically.

Security model for SQL agent:

- Regex validation blocks any query not starting with SELECT

- Forbidden keywords: INSERT, UPDATE, DELETE, DROP, ALTER, TRUNCATE, CREATE, GRANT, REVOKE, COPY, CALL
- Semicolon detection prevents multi-statement injection
- Explicit schema provided to LLM — only known tables can be referenced
- LIMIT automatically appended if missing — maximum 100 rows returned

Deterministic Intelligence Fallback

This is the feature I'm most proud of engineering. When no LLM API key is set, the `_fallback()` function in `decision.py` answers 16+ distinct question categories correctly from the pre-fetched evidence dictionary. The routing uses precise keyword matching with priority ordering to avoid ambiguity.

Category	Example Question	Answer Source
Risk analysis	Which customers are at risk and why?	risk_analysis list, risk_score, revenue_drop
Total revenue	What is the total revenue?	summary.total_revenue
Industry revenue	Which industry generated most revenue?	industry_summary[0] sorted by revenue
Industry customers	Which industries have most customers?	industry_summary sorted by count
Top customers	Show top 5 customers by revenue	top_customers[:5]
Least revenue	Which customer generated least revenue?	top_customers[-1]
Customer count	How many customers do we have?	summary.customers
Support breakdown	Support breakdown by priority	support_breakdown list
High-priority	High-priority tickets open?	summary.high_priority_tickets
Support load	What is the current support load?	summary.open_tickets
Business snapshot	What is the business health?	Full summary combined
Revenue trend	What is the revenue trend?	Context + note about time-series
Average sale	What is the average sale amount?	Revenue / customers with note

07 · FRONTEND APPLICATION

Frontend Application

The frontend is a Next.js 14 application using the App Router. It is a single-page experience with three sections: the AI question interface, the CSV data import panel, and the live results dashboard.

Design System

Built entirely with vanilla CSS custom properties — no Tailwind, no CSS framework. The aesthetic is a dark neon palette with electric cyan and deep purple accents. Design principles: glassmorphism cards, smooth micro-animations, hover states on every interactive element, and responsive layouts down to mobile viewports.

Key UX Features

- **Enter to submit:** pressing Enter in the question textarea submits immediately; Shift+Enter inserts a newline
- **Auto-scroll to answer:** after an answer arrives, the page smoothly scrolls to the answer block
- **Question guide:** 30+ categorised example questions toggle open/closed with animation
- **One-click question loading:** clicking any example pre-fills the textarea and scrolls to the input
- **Evidence trace panel:** the raw JSON evidence used to produce each answer is displayed live
- **CSV mapper:** drag-and-drop upload with column inspection, fuzzy mapping, and import flow
- **Live stats dashboard:** customer count, total revenue, open tickets, high-priority count
- **Show all customers:** top-customers table expands from 6 to full list on demand

State Management

All state is local React **useState** hooks — no Redux, no Zustand, no Context. The application is simple enough that global state management would be over-engineering. Server state is fetched fresh on load via **useEffect** and after each CSV import.

Environment Configuration

```
# frontend/.env.local
NEXT_PUBLIC_API_URL=http://localhost:8000

# The NEXT_PUBLIC_ prefix is required – Next.js only exposes
# env vars with this prefix to the browser bundle at build time.
# Without this file the variable would be undefined in the client.
```

Database Design

Schema Overview

Table	Columns	Purpose
customers	id, name, industry, annual_value	Master customer record
sales	id, customer_id, amount, sale_date	Individual sale transactions
support_tickets	id, customer_id, priority, status, subject, created_at	Customer support requests
documents	id, source, page, content, embedding (vector 384), created_at	RAG document chunks + vectors

Key SQL Patterns

```
-- Customer risk: 90-day revenue vs prior 90-day window
SELECT COALESCE(SUM(amount), 0) FROM sales
WHERE customer_id = %s
AND sale_date >= CURRENT_DATE - INTERVAL '90 days'

-- Industry revenue aggregation (requires explicit alias for ORDER BY)
SELECT c.industry,
COUNT(DISTINCT c.id),
COALESCE(SUM(s.amount), 0) AS revenue -- alias is required!
FROM customers c LEFT JOIN sales s ON s.customer_id = c.id
GROUP BY c.industry ORDER BY revenue DESC

-- Vector similarity search (cosine distance)
SELECT source, page, content,
1 - (embedding <=> %s::vector) AS score
FROM documents WHERE embedding IS NOT NULL
ORDER BY embedding <=> %s::vector LIMIT %s
```

Bug fixed during development: The industry revenue query originally used **ORDER BY revenue DESC** without an explicit column alias. PostgreSQL requires the alias to exist in the SELECT clause for ORDER BY to reference it by name. SQLite allows it without the alias, masking the bug in tests. Adding **AS revenue** fixed the production crash and made 3 integration tests pass.

API Reference

Business Intelligence Endpoints

GET /api/v1/business/summary

Description: Dashboard KPI snapshot

Request: No body

Response: { "customers": 1310, "total_revenue": 1606000.6, "open_tickets": 14, "high_priority_tickets": 2 }...

GET /api/v1/business/top-customers?limit=10

Description: Revenue-ranked customer list

Request: Query param: limit (1-50)

Response: { "customers": [{ "customer_id": 10, "customer": "UrbanCart", "industry": "E-commerce", "revenue": 233383.6 }] }...

POST /api/v1/business/ask

Description: Full intelligence pipeline

Request: { "question": "Which customers are at risk?" }

Response: { "answer": "...", "evidence": { "intent": "business", "business": { ... } } }...

POST /api/v1/business/risk

Description: Customer risk assessment

Request: { "customer_id": 3 }

Response: { "risk_score": 86, "risk_level": "HIGH", "revenue_drop_percent": 48.6 }...

Data Import Endpoints

POST /api/v1/data/analyze — Upload a CSV file and receive the detected dataset type, auto-suggested column mapping, missing fields, and a 5-row sample. No data is written.

POST /api/v1/data/upload — Upload CSV with confirmed column mapping. Optionally truncate existing data with `replace=true`. Supports multi-format date parsing (YYYY-MM-DD, DD/MM/YYYY, MM/DD/YYYY).

10 · SECURITY ENGINEERING

Security Engineering

For a project that takes free-text input and sends it to an LLM and a SQL database, security is not optional. Business Signal has three layers of protection.

1. Prompt Injection Guard

Every question passes through `sanitize_question()` in `prompt_guard.py` before touching the LLM or SQL agent. Four regex patterns detect common injection attempts:

```
INJECTION_PATTERNS = [
    r'ignore\s+(all\s+)?previous instructions',
    r'reveal\s+(the\s+)?system prompt',
    r'show\s+(me\s+)?your\s+hidden instructions',
    r'disregard\s+the\s+rules',
]

def sanitize_question(text: str) -> str:
    if is_suspicious(text): # case-insensitive regex match
        raise ValueError('Question rejected by prompt-injection guard')
    return text.strip()
```

2. SQL Injection Prevention

The SQL agent has a multi-layer defence against SQL injection and data exfiltration:

- All queries validated with regex to start with SELECT
- Forbidden keywords regex blocks all write/DDL operations (11 keywords)
- Semicolon check prevents multi-statement injection
- Only the known schema is provided to the LLM — unknown tables are unreferenceable
- LIMIT 100 enforced on all results to prevent data dumping
- Parameterised queries used everywhere in the business logic layer

3. CORS Configuration

Cross-Origin Resource Sharing is explicitly configured to only allow the frontend origins. The allowed origins list is configurable via the **CORS_ORIGINS** environment variable, defaulting to localhost ports 3000 and 3050.

```
app.add_middleware(CORSMiddleware,
    allow_origins=cors_origins,
    allow_methods=['GET', 'POST', 'PUT', 'DELETE', 'OPTIONS'],
    allow_headers=['*'],
    allow_credentials=False,
)
```

Testing & Evaluation

21 / 21 Unit tests pass	4 / 4 Eval tests pass	0 Errors or warnings	7.66s Full suite runtime
----------------------------	--------------------------	-------------------------	-----------------------------

Unit Test Coverage

Test file	Tests	What is verified
test_prompt_guard.py	4	Normal questions pass, 3 injection patterns detected
test_sql_agent.py	5	SELECT allowed, 4 write/DDL variants blocked
test_sql_security.py	4	Duplicate SQL security coverage in backend/tests/
test_data_api.py	2	Alternate column name aliases resolve correctly
test_intelligence.py	3	Risk context, revenue context, summary evidence all built

Evaluation Harness

Beyond unit tests, Business Signal has a separate evaluation suite in **evaluation/run.py** that tests the full pipeline end-to-end against a live Postgres database:

```
# evaluation/run.py - 4 deterministic risk assessments
Q001: customer_id=3 (Apex Logistics) → expected HIGH ✓ PASS
Q002: customer_id=7 (Pulse Media) → expected HIGH ✓ PASS
Q003: customer_id=1 (Nova Retail) → expected LOW ✓ PASS
Q004: customer_id=2 (Vertex Systems) → expected LOW ✓ PASS

Result: 4/4 passed
```

GitHub Actions CI

Every push to the repository triggers an automated pipeline that spins up a PostgreSQL service container, runs all pytest tests, and runs the evaluation harness. This ensures no regression is introduced unnoticed.

Data Ingestion Pipeline

Business Signal accepts CSV uploads for customers, sales, and support tickets. The ingestion pipeline handles real-world messiness: non-standard column names, mixed date formats, customer name lookups instead of IDs.

Column Alias Resolution

The mapper in `data_api.py` normalises column names (lower, strip, alphanumeric only) and checks against 40+ pre-defined alias sets. This means a file with columns named *Company Name*, *ARR*, and *Vertical* will map correctly to *name*, *annual_value*, and *industry* automatically.

Target field	Accepted column name variants (partial list)
name	customer, company, account, organization, client_name
annual_value	arr, acv, contract_value, yearly_revenue, customer_value
sale_date	date, closed_at, transaction_date, order_date, invoice_date
customer_id	account_id, client_id, company_id, customer_name, company
priority	severity, urgency, ticket_priority
status	ticket_status, state, case_status

Date Format Support

The `_parse_date()` helper tries four common formats in order, giving informative error messages when none match:

```
Supported date formats (tried in order):
YYYY-MM-DD (ISO 8601, preferred)
DD/MM/YYYY (European)
MM/DD/YYYY (US)
DD-MM-YYYY (hyphenated European)
```

Customer Reference Resolution

When importing sales or tickets, the `customer_id` field can be either a numeric ID or a customer name string. `_customer_ref()` tries the numeric parse first, then falls back to a case-insensitive name lookup. This allows importing files that reference customers by name, not just by database ID.

Deployment Architecture

The project ships with a Docker Compose configuration that makes local setup a single command. The architecture is intentionally minimal — two containers plus a local Next.js process.

```
# docker-compose.yml
services:
  postgres:
    image: pgvector/pgvector:pg17
    environment:
      POSTGRES_DB: business_signal
      POSTGRES_USER: business_signal
      POSTGRES_PASSWORD: business_signal
    ports: ['5432:5432']
    healthcheck:
      test: ['CMD-SHELL', 'pg_isready -U business_signal']

  api:
    build: ./backend
    environment:
      DATABASE_URL: postgresql://...
      LLM_API_KEY: ${LLM_API_KEY:-}
    ports: ['8000:8000']
    depends_on:
      postgres: { condition: service_healthy }
```

One-Command Setup

```
# 1. Start backend + database
docker compose up --build

# 2. Start frontend (separate terminal)
cd frontend && npm install && npm run dev

# 3. Open browser
http://localhost:3050

# Optional: enable Gemini AI answers
# Add LLM_API_KEY=your_gemini_key to .env
```

Port Configuration

Service	Port	Note
PostgreSQL	5432	Docker-mapped, only accessible from API container and localhost
FastAPI API	8000	Docker-mapped, accessible from browser and Next.js frontend
Next.js UI	3050	Local dev server (3000 reserved by Windows on this machine)

Performance & Scalability

Response Time Profile

Operation	Typical latency	Bottleneck
GET /summary	< 10ms	Single SQL aggregation
GET /top-customers	< 20ms	JOIN + ORDER BY
POST /ask (no LLM)	50–200ms	50 customer risk calculations
POST /ask (with Gemini)	1–4s	Gemini network round-trip
POST /data/analyze	< 100ms	CSV parse + mapping check
POST /data/upload (1k rows)	200–500ms	executemany batch insert
PDF ingest (10 pages)	2–5s	Sentence-Transformer encoding
Vector search	< 30ms	pgvector cosine scan

Scalability Considerations

- **Sentence-Transformer caching:** the embedding model is loaded once on first use and cached in-process — no reload per request
- **Risk calculation at scale:** the current loop over all customers is $O(n)$. At 10,000+ customers, precomputing risk on a schedule would be needed
- **pgvector indexing:** for large document collections an IVFFlat or HNSW index on the embedding column should be created
- **Connection pooling:** psycopg3 supports async connection pools — the current synchronous model suits this demo's concurrency needs
- **LLM cost:** Gemini Flash is used specifically for its low cost and high speed — evidence-first design minimises token usage by pruning context

Future Roadmap

Business Signal is a project showcase, not a production product. These are the extensions that would make logical next steps if taken to production:

01	Authentication & Multi-tenancy Add JWT-based authentication so multiple businesses can each have isolated data. Row-level security in PostgreSQL would enforce data isolation cleanly.
02	Streaming Answers Use Gemini's streaming API to show the answer token-by-token in the UI. Server-Sent Events or WebSockets from FastAPI to Next.js.
03	Scheduled Risk Alerts Background job (APScheduler or Celery) that computes risk scores nightly and sends email/Slack alerts when accounts cross the HIGH threshold.
04	Excel / Google Sheets Import Extend the data ingestion pipeline to accept .xlsx files and Google Sheets URLs, removing the CSV conversion step for most business users.
05	Revenue Forecasting Add a time-series forecasting module (Prophet or statsmodels) to project revenue 30/60/90 days forward per customer and per industry.
06	Interactive Charts Add Chart.js or Recharts to the frontend to visualise the revenue trend, risk distribution, and industry breakdown as dynamic charts.
07	pgvector HNSW Index Migrate document embeddings to an HNSW approximate nearest-neighbour index for sub-millisecond vector search at millions of document chunks.
08	LLM Function Calling Replace the keyword classifier with Gemini function-calling to let the LLM choose which tools to invoke — a step toward a proper agent architecture.

Conclusion

Business Signal started with a question: *what does a responsible AI system that answers business questions actually look like?* Not a wrapper around a chat API that makes up numbers — but a deliberate pipeline where every answer is earned through evidence.

The core insight this project demonstrates: **AI engineering is not about prompting. It is about designing the pipeline that decides what evidence the model sees, validating that evidence is correct, and ensuring the model cannot stray beyond it.**

What I learned and demonstrated:

- How to build a multi-source evidence pipeline that routes questions to the right tool
- How pgvector turns PostgreSQL into a production-ready vector store without a separate service
- How to design a deterministic fallback so the system is useful without any API key
- How to protect an LLM-integrated system against prompt injection at the boundary
- How to validate SQL generated by an LLM so it can never write, drop, or exfiltrate data
- How to structure a FastAPI backend with clear module boundaries and a testable connection layer
- How to build a polished Next.js frontend with only vanilla CSS and no component library
- How to configure Docker Compose health checks so the API never starts before the database is ready
- How to write an evaluation harness that tests the full pipeline, not just individual functions
- The practical difference between grounded AI answers (citing evidence) and hallucinated ones

Final Metrics

13+	16+	21/21	8+
Backend modules	API endpoints	Tests passing	Bug fixes shipped

Thank you for reading. The full source code, commit history, and GitHub Actions pipeline are available at:

github.com/ranjiths112007/Business-Signal-enterprise-ai